

Relational calculus (Tuple Relational Calculus and Domain Relational Calculus) is an alternative to relational algebra. In contrast to the algebra, which is procedural, the calculus is nonprocedural, or declarative, in that it allows us to describe the set of answers without being explicit about how they should be computed.

what

Tuple Relational Calculus

A tuple variable is a variable that takes on tuples of a particular relation schema as values. That is, every value assigned to a given tuple variable has the same number and type of fields. A tuple relational calculus query has the form $\{ T \mid p(T) \}$, where T is a tuple variable and $p(T)$ denotes a formula that describes T ; we will shortly define formulas and queries rigorously. The result of this query is the set of all tuples t for which the formula $p(T)$ evaluates to **true** with $T = t$.

Consider the following schema:

Sailors(sid: integer, sname: string, rating: integer, age: real)
 Boats(bid: integer, bname: string, color: string)
 Reserves(sid: integer, bid: integer, day: date)

The key fields are underlined, and the domain of each field is listed after the field name. Thus *sid* is the key for Sailors, *bid* is the key for Boats, and all three fields together form the key for Reserves. Fields in an instance of one of these relations will be referred to by name, or positionally, using the order in which they are listed above.

<i>bid</i>	<i>bname</i>	<i>color</i>
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Figure 4.17 An Instance *B1* of Boats

<i>sid</i>	<i>sname</i>	<i>rating</i>	<i>age</i>
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Figure 4.15 An Instance *S3* of Sailors

<i>sid</i>	<i>bid</i>	<i>day</i>
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Figure 4.16 An Instance *R2* of Reserves

Find all sailors with a rating above 7.

$\{ \tau \mid p(\tau) \}$

$\{ S \mid S \in \text{Sailors} \wedge S.\text{rating} > 7 \}$

Find all sailors with a rating above 7.

27/10/17

$$\{S \mid S \in \text{Sailors} \wedge S.\text{rating} > 7\}$$

When this query is evaluated on an instance of the Sailors relation, the tuple variable S is instantiated successively with each tuple, and the test $S.\text{rating} > 7$ is applied. The answer contains those instances of S that pass this test. On instance $S3$ of Sailors, the answer contains Sailors tuples with sid 31, 32, 58, 71, and 74.

Syntax of TRC Queries

We now define these concepts formally, beginning with the notion of a formula. Let Rel be a relation name, R and S be tuple variables, a an attribute of R , and b an attribute of S . Let op denote an operator in the set $\{<, >, =, \leq, \geq, \neq\}$. An **atomic formula** is one of the following:

- $R \in \text{Rel}$ ✓
- $R.a \text{ op } S.b$ ✓
- $R.a \text{ op } \text{constant}$, or $\text{constant op } R.a$ ✓

A **formula** is recursively defined to be one of the following, where p and q are themselves formulas, and $p(R)$ denotes a formula in which the variable R appears:

- any atomic formula
- $\neg p$, $p \wedge q$, $p \vee q$, or $p \Rightarrow q$
- $\exists R(p(R))$, where R is a tuple variable
- $\forall R(p(R))$, where R is a tuple variable

In the last two clauses above, the **quantifiers** $\exists$ and $\forall$ are said to **bind** the variable R . A variable is said to be **free** in a formula or subformula (a formula contained in a larger formula) if the (sub)formula does not contain an occurrence of a quantifier that binds it.

A **TRC query** is defined to be expression of the form $\{T \mid p(T)\}$, where T is the only free variable in the formula p .

Examples of TRC Queries

We now illustrate the calculus through several examples, using the instances *B1* of Boats, *R2* of Reserves, and *S3* of Sailors shown in Figures 4.15, 4.16, and 4.17. We will use parentheses as needed to make our formulas unambiguous. Often, a formula $p(R)$ includes a condition $R \in Rel$, and the meaning of the phrases *some tuple* R and *for all tuples* R is intuitive. We will use the notation $\exists R \in Rel(p(R))$ for $\exists R(R \in Rel \wedge p(R))$. Similarly, we use the notation $\forall R \in Rel(p(R))$ for $\forall R(R \in Rel \Rightarrow p(R))$.

$$\exists R \in Rel(p(R))$$

$$\exists R (R \in Rel \wedge p(R))$$

$$\forall R \in Rel(p(R))$$

$$\forall R (R \in Rel \Rightarrow p(R))$$

$$P \Rightarrow Q$$

T	T	✓
T	F	✗
F	F	✓
F	T	✓

Find the names and ages of sailors with a rating above 7.

$$\{P \mid \exists S \in Sailors(S.rating > 7 \wedge \underline{P.name = S.sname} \wedge \underline{P.age = S.age})\}$$

↑

This query illustrates a useful convention: P is considered to be a tuple variable with exactly two fields, which are called *name* and *age*, because these are the only fields of P that are mentioned and P does not range over any of the relations in the query; that is, there is no subformula of the form $P \in Relname$. The result of this query is a relation with two fields, *name* and *age*. The atomic formulas $P.name = S.sname$ and $P.age = S.age$ give values to the fields of an answer tuple P . On instances *B1*, *R2*, and *S3*, the answer is the set of tuples $\langle Lubber, 55.5 \rangle$, $\langle Andy, 25.5 \rangle$, $\langle Rusty, 35.0 \rangle$, $\langle Zorba, 16.0 \rangle$, and $\langle Horatio, 35.0 \rangle$.

<i>sid</i>	<i>sname</i>	<i>rating</i>	<i>age</i>
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Figure 4.15 An Instance *S3* of Sailors

Sailors(*sid*: integer, *sname*: string, *rating*: integer, *age*: real)
 Boats(*bid*: integer, *bname*: string, *color*: string)
 Reserves(*sid*: integer, *bid*: integer, *day*: date)

Find the sailor name, boat id, and reservation date for each reservation.

$\{P \mid \exists R \in \text{Reserves} \ \exists S \in \text{Sailors}$
 $(\underline{R.sid = S.sid} \wedge \underline{P.bid = R.bid} \wedge \underline{P.day = R.day} \wedge \underline{P.sname = S.sname})\}$

For each Reserves tuple, we look for a tuple in Sailors with the same *sid*. Given a pair of such tuples, we construct an answer tuple *P* with fields *sname*, *bid*, and *day* by copying the corresponding fields from these two tuples. This query illustrates how we can combine values from different relations in each answer tuple. The answer to this query on instances *B1*, *R2*, and *S3* is shown in Figure 4.20.

<i>sid</i>	<i>sname</i>	<i>rating</i>	<i>age</i>
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Figure 4.15 An Instance *S3* of Sailors

<i>sid</i>	<i>bid</i>	<i>day</i>
22	101	10/10/98
22	102	10/10/98
✓ 22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
✓ 31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
✓ 74	103	9/8/98

Figure 4.16 An Instance *R2* of Reserves

<i>sname</i>	<i>bid</i>	<i>day</i>
Dustin	101	10/10/98
Dustin	102	10/10/98
Dustin	103	10/8/98
Dustin	104	10/7/98
Lubber	102	11/10/98
Lubber	103	11/6/98
Lubber	104	11/12/98
Horatio	101	9/5/98
Horatio	102	9/8/98
Horatio	103	9/8/98

Figure 4.20 Answer to Query

(*Q1*) Find the names of sailors who have reserved boat 103.

$\{P \mid \exists S \in \text{Sailors} \ \exists R \in \text{Reserves}(\underline{R.sid = S.sid} \wedge \underline{R.bid = 103} \wedge \underline{P.sname = S.sname})\}$

This query can be read as follows: “Retrieve all sailor tuples for which there exists a tuple in Reserves, having the same value in the *sid* field, and with *bid* = 103.” That is, for each sailor tuple, we look for a tuple in Reserves that shows that this sailor has reserved boat 103. The answer tuple *P* contains just one field, *sname*.

(Q2) Find the names of sailors who have reserved a red boat.

$$\{P \mid \exists S \in \text{Sailors} \ \exists R \in \text{Reserves} (R.sid = S.sid \wedge P.sname = S.sname \\ \wedge \exists B \in \text{Boats} (B.bid = R.bid \wedge B.color = 'red'))\}$$

This query can be read as follows: “Retrieve all sailor tuples S for which there exist tuples R in Reserves and B in Boats such that $S.sid = R.sid$, $R.bid = B.bid$, and $B.color = 'red'$.” Another way to write this query, which corresponds more closely to this reading, is as follows:

$$\{P \mid \exists S \in \text{Sailors} \ \exists R \in \text{Reserves} \ \exists B \in \text{Boats} \\ (\underbrace{R.sid = S.sid} \wedge \underbrace{B.bid = R.bid} \wedge \underbrace{B.color = 'red'} \wedge \underbrace{P.sname = S.sname})\}$$

Safety of Expressions

A tuple-relational-calculus expression may generate an infinite relation.

Suppose that we write the expression:

$$\{t \mid \neg (t \in \text{instructor})\}$$

$\neg (t \in \text{instructor})$
 $P(t)$

The expression $\{t \mid \neg (t \in \text{instructor})\}$ is not safe.

There are infinitely many tuples that are not in instructor. Most of these tuples contain values that do not even appear in the database! We do not wish to allow such expressions.

Domain Relational Calculus

A domain variable is a variable that ranges over the values in the domain of some attribute (e.g., the variable can be assigned an integer if it appears in an attribute whose domain is the set of integers). A DRC query has the form $\{\langle x_1, x_2, \dots, x_n \rangle \mid p(\langle x_1, x_2, \dots, x_n \rangle)\}$, where each x_i is either a domain variable or a constant and $p(\langle x_1, x_2, \dots, x_n \rangle)$ denotes a **DRC formula** whose only free variables are the variables among the x_i , $1 \leq i \leq n$. The result of this query is the set of all tuples $\langle x_1, x_2, \dots, x_n \rangle$ for which the formula evaluates to **true**.

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid p(\langle x_1, x_2, \dots, x_n \rangle) \}$$

Sailors(sid: integer, sname: string, rating: integer, age: real)

Boats(bid: integer, bname: string, color: string)

Reserves(sid: integer, bid: integer, day: date)

$\langle I, N, T, A \rangle$

Find all sailors with a rating above 7.

$$\{\langle I, N, T, A \rangle \mid \langle I, N, T, A \rangle \in \text{Sailors} \wedge \underline{T > 7}\}$$

This differs from the TRC version in giving each attribute a (variable) name. The condition $\langle I, N, T, A \rangle \in \text{Sailors}$ ensures that the domain variables I , N , T , and A are restricted to be fields of the *same* tuple. In comparison with the TRC query, we can say $T > 7$ instead of $S.\text{rating} > 7$, but we must specify the tuple $\langle I, N, T, A \rangle$ in the result, rather than just S .

EXPRESSIVE POWER OF ALGEBRA AND CALCULUS

We have presented two formal query languages for the relational model. Are they equivalent in power? Can every query that can be expressed in relational algebra also be expressed in relational calculus? The answer is yes, it can. Can every query that can be expressed in relational calculus also be expressed in relational algebra? Before we answer this question, we consider a major problem with the calculus as we have presented it.

Consider the query $\{S \mid \neg(S \in Sailors)\}$. This query is syntactically correct. However, it asks for all tuples S such that S is not in (the given instance of) Sailors. The set of such S tuples is obviously infinite, in the context of infinite domains such as the set of all integers. This simple example illustrates an *unsafe* query. It is desirable to restrict relational calculus to disallow unsafe queries.

Returning to the question of expressiveness, we can show that every query that can be expressed using a *safe* relational calculus query can also be expressed as a relational algebra query. The expressive power of relational algebra is often used as a metric of how powerful a relational database query language is. If a query language can express all the queries that we can express in relational algebra, it is said to be **relationally complete**.